Elastic Deformation of Two-Dimensional Objects

Combined working draft (sections drafted so far)

Group 02

November 10, 2026

1 Building the model

We study how the geometry of a two-dimensional elastic object and its supports affect its response to applied forces. Our unknown is the displacement $\mathbf{u}(x, y)$, which tells us how far each point moves horizontally and vertically. Differences in movement between nearby points describe local stretching and shearing, called strain. The material resists these changes through internal forces per unit area, called stress. We assume small deformations and a linear relation between strain and stress, leaving the material choice to the individual cases [3].

We look for the shape after the object has settled under the load. Forces must then balance in both directions throughout the material:

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} + g_x = 0, \qquad \frac{\partial \sigma_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} + g_y = 0. \tag{1}$$

Here the σ terms are stress components and g_x, g_y are applied forces per unit volume. Since stress depends on strain, and strain depends on displacement, Eq. (1) gives equations for $\mathbf{u}$. We also specify where forces act on the boundary and which movements the supports prevent.

To make this a finite problem, we divide the region into filled triangles as shown in Fig. 1, following its boundary and leaving any holes empty [11]. Their vertices are the mesh nodes, each with two unknown displacement components. Inside each triangle, displacement varies linearly between its three vertex values. Adjacent triangles share these values, so their displacements agree along the common edge. This represents movement throughout the solid; the triangle edges are not physical bars. Smaller triangles allow us to capture more variation in the deformation.

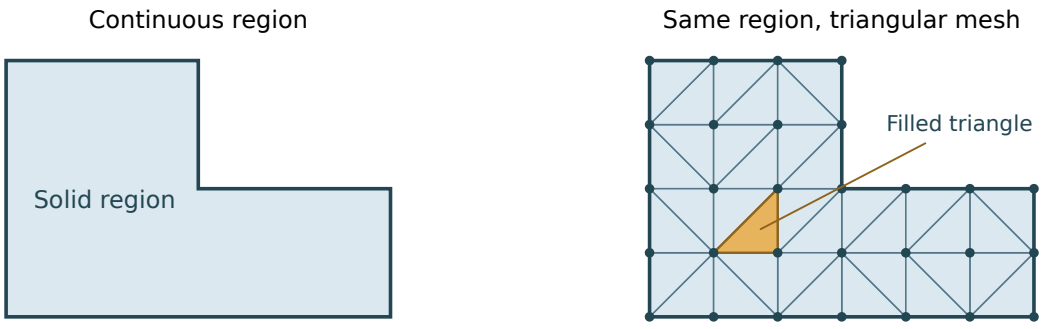


Figure 1: Triangulation of an illustrative region. The filled triangles cover the same solid area and share nodes along their edges. Displacement is approximated throughout each triangle using its three vertex values. The outline is an example, not a selected project geometry.

The finite element method applies Eq. (1) by balancing internal and external work, meaning forces acting through small allowed displacements. Using the triangle approximation and adding

contributions at shared nodes gives

$$Kd = f + r, \tag{2}$$

where d collects node displacements, f the applied forces and r the forces exerted by supports. The stiffness matrix K describes how the mesh and material resist displacement. Most entries are zero because only nodes in a common triangle interact directly.

Finally, we separate unknown displacements (F) from values prescribed at supports (C). The free rows of Eq. (2) become

$$Ax = b. \tag{3}$$

Here $A = K_{FF}$, $x = d_F$ and $b = f_F - K_{FC}d_C$; for fixed supports, $d_C = 0$ and $b = f_F$. Supports must prevent each separate piece from freely sliding or turning. Solving Eq. (3) then gives the displacements used to draw the deformed object.

2 Displacement, forces and matrix construction

The preceding section introduced $Ax = b$ as the system that determines how the object moves. We now build it by describing the material's response, calculating each triangle's contribution and applying the supports.

2.1 From displacement to internal forces

Our working model is a thin sheet of constant thickness $t > 0$, with the same material properties everywhere and in every direction. Forces act within its plane, and its front and back faces are free of applied forces. We use *plane stress*: stresses through the thickness are neglected, although the thickness may change. We consider the settled shape under static loading, with strains and rotations small enough for a linear approximation. The model excludes bending and permanent deformation.

The material resists stretching and changes of angle. For displacement $\mathbf{u} = (u_x, u_y)^T$, these changes form the strain vector

$$\varepsilon(\mathbf{u}) = \begin{bmatrix} \partial u_x/\partial x \\ \partial u_y/\partial y \\ \partial u_x/\partial y + \partial u_y/\partial x \end{bmatrix}, \qquad \sigma = D\varepsilon, \qquad D = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}. \tag{4}$$

The first two entries measure stretching in the coordinate directions; the third measures the change of a right angle, called engineering shear. The stress vector $\sigma = (\sigma_{xx}, \sigma_{yy}, \sigma_{xy})^T$ contains the corresponding internal forces per unit area. Young's modulus $E > 0$ controls resistance to stretching, while Poisson's ratio $-1 < \nu < 1/2$ describes the sideways response [1]. Lengths and displacements are in metres, strain is dimensionless, and stress and E are in pascals. Each case specifies the material values.

2.2 From triangles to the stiffness matrix

Inside triangle e, we write displacement as $\mathbf{u}_h = \sum_{i=1}^{3} N_i \mathbf{u}_i$. Each linear weight N_i is one at vertex i and zero at the other vertices. Differentiating these weights gives a matrix B_e that converts the triangle's six vertex displacements, collected in d_e, into strain: $\varepsilon(\mathbf{u}_h) = B_e d_e$. The derivatives are constant, so strain is constant within each triangle.

We express force balance by equating internal and external work for any small test movement allowed by the supports. Substituting the triangle approximation gives its stiffness matrix

$$K_e = \int_{T_e} tB_e^T D B_e \, dA = tS_e B_e^T D B_e, \tag{5}$$

where T_e is the triangle and $S_e > 0$ its area. This 6×6 matrix converts vertex movements into internal nodal forces [14]. We add its entries to the corresponding rows and columns of K, using the displacement order $d = (u_{x,0}, u_{y,0}, \ldots, u_{x,n-1}, u_{y,n-1})^T$ for n nodes. Contributions at shared nodes are added together. Nodes without a common triangle have no direct interaction, so most entries are zero and we store K sparsely.

For general objects, we plan to mesh ordered outer and hole boundaries using Python's `triangle` interface to Triangle [11]. Current rectangular examples instead use rectangles split into triangles. Meshes must have positive-area triangles, matching shared edges and empty holes. The current assembly code is `fem/elasticity.py:assemble`.[1]

External forces are distributed to nodes using the same weights N_i. For example, a constant boundary force per unit area $\bar{\tau}$, called traction, gives a force per unit edge length $q = t\bar{\tau}$. An edge of length ℓ contributes $\ell q/2$ to each endpoint [14]. Forces acting throughout the material are integrated over each triangle. The resulting load vector f is in newtons and K is in N/m. Thickness enters each construction once. Refinement must preserve the physical loaded boundary and its applied traction.

2.3 From supports to a solvable system

The supports prescribe displacement components indexed by C; the remaining components, indexed by F, are unknown. A support can prevent movement in one direction while allowing it in the other. Taking the free rows of $Kd = f + r$ and moving known terms to the right gives

$$\underbrace{K_{FF}}_{A}\underbrace{d_F}_{x} = \underbrace{f_F - K_{FC}d_C}_{b}. \tag{6}$$

The subscripts select rows and columns, respectively. This recovers $Ax = b$ from Section 1; fixed supports have $d_C = 0$ and $b = f_F$. After solving, the forces exerted by supports, called reactions, are

$$r_C = K_{CF}d_F + K_{CC}d_C - f_C. \tag{7}$$

We retain the original loads, including any applied at supports, to calculate these reactions. Applied forces and reactions should together balance total force and moment.

The construction also explains why supports are essential. Since D is symmetric and positive definite, K is symmetric and

$$d^T K d = \sum_e t S_e (B_e d_e)^T D (B_e d_e) \geq 0. \tag{8}$$

Half this quantity is the stored elastic energy. An unsupported piece can slide or rotate without straining, giving zero energy and making K singular. Supports must prevent both translations and rotation on every piece. For positive-area triangles connected across shared edges, removing these motions makes the energy positive for every nonzero free displacement. Thus A is symmetric positive definite. Three arbitrary constraints need not suffice; pieces joined only at a node require additional care.

Even with a unique solution, small load changes may affect the computed movement. For the supported matrix, the condition number is

$$\kappa_2(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}, \tag{9}$$

where the numerator and denominator are its largest and smallest eigenvalues. A large ratio means a small percentage change in load can cause a much larger percentage change in displacement. The effect depends on the direction of the load change. We will estimate these eigenvalues

[1] The course-provided routine has not been located; permission to substitute the team's implementation remains unconfirmed.

using dense routines for small systems and sparse `eigsh` for larger ones [7], checking convergence and whether the estimates satisfy their eigenvalue equations. Geometry, mesh and supports can all affect conditioning. These properties guide the solver choices and accuracy checks in the next section.

3 Numerical methods and comparison protocol

Section 2 showed that the supported matrix A is sparse, symmetric and positive definite, with sensitivity measured by $\kappa_2(A)$. We now use these properties to choose solvers and fix one protocol for comparing them. Let $N = |F|$ be the number of unknowns, x_k the k-th iterate, $e_k = x - x_k$ its error and $r_k = b - Ax_k$ its residual.

3.1 Direct methods

A direct method factorizes A into triangular factors: $PA = LU$ with a row permutation P, or, for symmetric positive definite A, Cholesky $A = LL^T$ without pivoting at half the cost [13]. Dense factorization costs $O(N^3)$ operations but each later solve only $O(N^2)$; forming the inverse costs more and is less accurate. In sparse A, elimination creates *fill-in*, nonzeros of the factors where A has zeros. SciPy's `splu` limits it with a column ordering (COLAMD) before LU with partial pivoting [10]; factorization dominates the cost, and each solve is two triangular substitutions.

3.2 Stationary iterations

Iterative methods need only products with A. A stationary method corrects the iterate with the residual, using an easily inverted approximation M of A:

$$x_{k+1} = x_k + M^{-1}(b - Ax_k), \qquad e_{k+1} = (I - M^{-1}A)\, e_k. \tag{10}$$

It converges for every x_0 if and only if the spectral radius $\rho(I - M^{-1}A)$ is below one [6]. *Jacobi* uses $M = D_A$, the diagonal of A. Its iteration matrix is similar to $I - S$, where $S = D_A^{-1/2} A D_A^{-1/2}$, so

$$\rho_J = \max_i |1 - \lambda_i(S)|. \tag{11}$$

Positive definiteness only gives $\lambda_i(S) > 0$, so Jacobi diverges when $\lambda_{\max}(S) > 2$. *Damped Jacobi*, $M = D_A/\omega$, converges if and only if $0 < \omega < 2/\lambda_{\max}(S)$; we use $\omega = 0.5$ and check this bound for each system. *Gauss–Seidel* takes M as the lower triangle of A, including the diagonal, so each sweep is one forward substitution. It converges for every symmetric positive definite A [6].

3.3 Krylov methods

For symmetric positive definite A, the solution of $Ax = b$ minimizes $\phi(y) = \frac{1}{2} y^T A y - b^T y$, the elastic energy of Eq. (8) minus the work of the loads, and $r = -\nabla\phi$. *Steepest descent* (SD) moves along r_k to the minimum of ϕ on that line. Each step multiplies its error in the energy norm $\|e\|_A = (e^T A e)^{1/2}$ by at most $(\kappa-1)/(\kappa+1)$, where $\kappa = \kappa_2(A)$ [12]. The *conjugate gradient* (CG) method instead minimizes $\|e_k\|_A$ over all of $x_0 + \mathcal{K}_k$, where $\mathcal{K}_k = \operatorname{span}\{r_0, Ar_0, \ldots, A^{k-1}r_0\}$ is the Krylov space. It needs one product with A per iteration and satisfies [12]

$$\|e_k\|_A \le 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_A, \tag{12}$$

so its iteration count grows with $\sqrt{\kappa}$ rather than κ. This matters because, as for the model Poisson problem, κ grows like h^{-2} on quasi-uniform meshes of size h [2].

Diagonally preconditioned CG (PCG) uses D_A as preconditioner and is equivalent to CG on S, so $\kappa_2(S)$ replaces κ in Eq. (12). If the diagonal is nearly constant, S is nearly a multiple of

A and nothing is gained. Two-dimensional element stiffness does not scale with element size, so the diagonal varies only with the number and shape of elements at a node and with material or thickness. *MINRES* minimizes $\|r_k\|_2$ over $x_0 + \mathcal{K}_k$ for symmetric A [4]; *GMRES* does so for general A [5], but must store and orthogonalize against all k basis vectors, costing $O(kN)$ memory and work at step k. For symmetric A both give the same iterates in exact arithmetic, so full GMRES (restart length N) measures the cost of ignoring symmetry. CG, PCG and MINRES exploit this structure; the other methods are baselines predicted by ρ_J and κ.

3.4 Comparison protocol

Every method receives the same A and b and starts from $x_0 = 0$. A returned vector $\hat{x}$ is accepted only if its recomputed relative residual satisfies

$$\eta(\hat{x}) = \frac{\|b - A\hat{x}\|_2}{\|b\|_2} < 10^{-8}. \tag{13}$$

Native solver flags are recorded but never replace this test, since SciPy's Krylov solvers stop on internal residual estimates [8, 9]; MINRES gets native tolerance zero and stops once Eq. (13) holds. An iteration is a sweep for the stationary methods, a search direction for SD, CG, PCG and MINRES, and an inner Arnoldi step for GMRES, which never restarts. The cap is 20,000 iterations, and ordinary Jacobi also stops if $\eta > 10^{10}$ or becomes non-finite. Hitting the cap or this divergence guard counts as failure; the status, iteration count and final η are still reported.

Accuracy is the relative error $\|\hat{x} - x_{\text{ref}}\|_2 / \|x_{\text{ref}}\|_2$ against the sparse LU solution x_{ref} of the same system. As $\hat{x}$ solves the system with load perturbed by its residual, the standard bound [13]

$$\frac{\|\delta x\|_2}{\|x\|_2} \le \kappa_2(A) \frac{\|\delta b\|_2}{\|b\|_2} \tag{14}$$

limits this error to $\kappa_2(A)\,\eta$. This *algebraic* error measures how well one mesh's equations are solved. *Discretization* error, between finite element and continuum displacements, does not shrink with the tolerance; we assess it by comparing meshes at matched physical points under the same physical load.

Times are the best of three wall-clock runs (`perf_counter`). Sparse LU factorization and solve are timed separately. Iterative times include method-specific setup and stopping checks, but exclude assembly, reduction, eigenvalue diagnostics, the reference solve and residual histories, which come from a separate run. They compare these SciPy implementations on one machine, not intrinsic algorithm costs. The small worked system also compares dense $PA = LU$, the explicit inverse and Cholesky. The eigenvalues in Eq. (9) come from dense `eigvalsh` for small systems, otherwise from `eigsh` with tolerance 10^{-10}: `which='LA'` for $\lambda_{\max}$ and shift-invert about zero for $\lambda_{\min}$ [7]. Applied to S, they give ρ_J and the damped-Jacobi bound.

Sections 4 and **??** also check that applied loads and the reactions of Eq. (7) have zero resultant force and moment, up to rounding, and that d_C equals the prescribed values. An affine patch test restrains the left edge only horizontally and the corner at the origin vertically, with uniform traction p on the opposite edge; the plane-stress solution $u_x = px/E$, $u_y = -\nu py/E$ is linear, so the computed nodal values must match it to rounding on any mesh. This verifies assembly, loading and reduction, not discretization error. Finally, the displacement change caused by a seeded random load perturbation is compared with the bound in Eq. (14).

4 Simple square demonstration

We now apply the construction of Section 2 and the protocol of Section 3 to the simplest controlled case: a square plate pulled uniformly at one end, whose expected stretch pL/E lets us check every computed number against physics.

The plate occupies $0 \le x, y \le L$ with $L = 1$ m and has thickness $t = 0.01$ m, $E = 200$ GPa and $\nu = 0.3$, in plane stress without body forces (Fig. 2a). The left edge is clamped, $u_x = u_y = 0$. The right edge carries the uniform traction $\bar{\tau} = (p, 0)$ with $p = 100$ MPa, so $q = tp = 10^6$ N/m with resultant $ptL = 10^6$ N; the top and bottom edges are free; the load is a prescribed traction, not a prescribed displacement. Units are m, Pa and N. The mesh has $m \times m$ square cells, each cut along its bottom-left to top-right diagonal, giving $(m+1)^2$ nodes and $2m^2$ triangles. The clamp prescribes $2(m+1)$ components, leaving $N = 2m(m+1)$ unknowns. The case is `force_driven` in `fem/plate.py`; all numbers below come from `draft/experiments/square.py`.

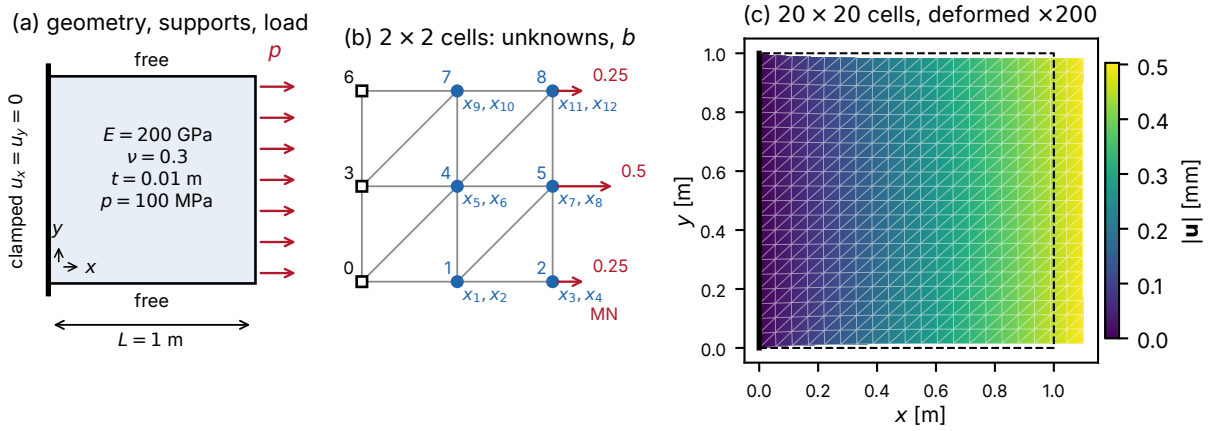


Figure 2: (a) Geometry, supports and load. (b) The 2×2-cell mesh: clamped nodes (squares), free nodes (dots) with their unknowns $x_1, \ldots, x_{12}$, and the nodal forces forming b. (c) Original outline (dashed) and deformed 20×20-cell plate, displacements amplified by 200; colour is the actual displacement magnitude in mm.

4.1 A small system solved directly

For $m = 2$ there are 9 nodes, 8 triangles and 18 displacement components. Clamping nodes 0, 3 and 6 leaves $N = 12$ unknowns, ordered node by node as $x = (u_{x,1}, u_{y,1}, u_{x,2}, \ldots, u_{y,8})^T$ (Fig. 2b). Since $d_C = 0$, $b = f_F$. Each loaded edge of length 0.5 m carries 0.5 MN, split equally between its end nodes, so $b = 2.5 \times 10^5\,(0,0,1,0,0,0,2,0,0,0,1,0)^T$ N. For $\nu = 0.3$ every entry of A is an integer multiple of $Et/(40(1-\nu^2)) = 5.495 \times 10^7$ N/m:

$$A = \frac{Et}{40(1-\nu^2)} \begin{bmatrix} 54 & -13 & -20 & 6 & -14 & 13 & \cdot & -13 & \cdot & \cdot & \cdot & \cdot \\ -13 & 54 & 7 & -7 & 13 & -40 & -13 & \cdot & \cdot & \cdot & \cdot & \cdot \\ -20 & 7 & 27 & -13 & \cdot & \cdot & -7 & 6 & \cdot & \cdot & \cdot & \cdot \\ 6 & -7 & -13 & 27 & \cdot & \cdot & 7 & -20 & \cdot & \cdot & \cdot & \cdot \\ -14 & 13 & \cdot & \cdot & 108 & -26 & -40 & 13 & -14 & 13 & \cdot & -13 \\ 13 & -40 & \cdot & \cdot & -26 & 108 & 13 & -14 & 13 & -40 & -13 & \cdot \\ \cdot & -13 & -7 & 7 & -40 & 13 & 54 & -13 & \cdot & \cdot & -7 & 6 \\ -13 & \cdot & 6 & -20 & 13 & -14 & -13 & 54 & \cdot & \cdot & 7 & -20 \\ \cdot & \cdot & \cdot & \cdot & -14 & 13 & \cdot & \cdot & 54 & -13 & -20 & 7 \\ \cdot & \cdot & \cdot & \cdot & 13 & -40 & \cdot & \cdot & -13 & 54 & 6 & -7 \\ \cdot & \cdot & \cdot & \cdot & \cdot & -13 & -7 & 7 & -20 & 6 & 27 & \cdot \\ \cdot & \cdot & \cdot & \cdot & -13 & \cdot & 6 & -20 & 7 & -7 & \cdot & 27 \end{bmatrix}. \tag{15}$$

Dots are zeros, mostly between nodes without a common triangle, such as 1 and 7. A is exactly symmetric, with eigenvalues from $\lambda_{\min} = 1.073 \times 10^8$ to $\lambda_{\max} = 1.028 \times 10^{10}$ N/m, so $\kappa_2(A) = 95.80$; the arrays are stored as CSV files in `draft/experiments/results/square/`.

Pivoted $PA = LU$, the explicit inverse and Cholesky $A = LL^T$ give relative residuals η (Eq. (13)) of 6.2×10^{-16}, 8.4×10^{-16} and 9.2×10^{-16}, and their solutions agree to 6.3×10^{-16} relative to their norm. From bottom to top, the right-edge nodes move $u_x = 0.5112$, 0.4822

and 0.4678 mm, close to $pL/E = 0.5$ mm, and $u_y = +0.1045$, $+0.0294$ and -0.0461 mm. The continuum problem is symmetric about $y = L/2$, which would make u_y antisymmetric; the deviation comes from the coarse mesh and its single diagonal direction.

4.2 A larger system and the iterative methods

For $m = 20$ the mesh has 441 nodes, 800 triangles and $N = 840$ unknowns. The sparse matrix stores 11,112 entries (9,590 numerically nonzero) and is symmetric up to rounding. Sparse estimates give $\lambda_{\min} = 1.917 \times 10^6$ and $\lambda_{\max} = 1.745 \times 10^{10}$ N/m, so $\kappa_2(A) = 9104$, in agreement with a dense computation to 10^{-14}. The scaled matrix S has eigenvalues in $[3.687 \times 10^{-4},\ 2.960]$, so $\rho_J = 1.960$ by Eq. (11). Table 1 and Fig. 3 report the methods.

Table 1: All methods on the 840-unknown system under the protocol of Section 3 (best of three times; Linux x86-64, Python 3.13, SciPy 1.18.1). An asterisk marks failure to reach $\eta < 10^{-8}$.

Method	Iterations	Final η	Error vs LU	Time (ms)	Stop
Sparse LU (`splu`)	–	3.5×10^{-14}	–	$2.20 + 0.09$	factor + solve
Jacobi*	49	1.3×10^{10}	3.4×10^{7}	0.65	divergence guard
Damped Jacobi ($\omega = 0.5$)*	20,000	4.8×10^{-6}	8.7×10^{-5}	284	iteration cap
Gauss–Seidel	19,136	1.0×10^{-8}	1.7×10^{-7}	7,178	target
Steepest descent*	20,000	3.1×10^{-6}	4.2×10^{-5}	541	iteration cap
CG	242	8.8×10^{-9}	9.6×10^{-10}	5.4	target
Diagonal PCG	226	8.8×10^{-9}	4.9×10^{-10}	6.8	target
MINRES	236	9.4×10^{-9}	5.9×10^{-9}	13.7	target
Full GMRES (restart $= N$)	228	9.5×10^{-9}	6.0×10^{-9}	186	target

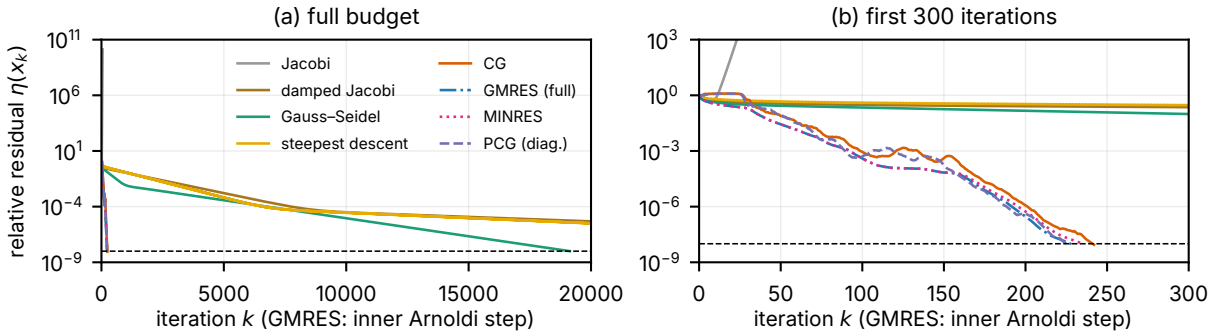


Figure 3: Relative residual $\eta(x_k)$ of every method on the 840-unknown system; the dashed line is the target 10^{-8}. (a) Whole budget. (b) First 300 iterations. GMRES shows its Arnoldi residual estimate; other curves are recomputed residuals.

The stationary methods behave as their diagnostics predict. Since $\lambda_{\max}(S) > 2$, Jacobi multiplies one error component by about -1.96 per sweep and is stopped by the guard after 49 sweeps. Damping with $\omega = 0.5$, below the limit $2/\lambda_{\max}(S) = 0.676$, restores convergence, but the slowest component shrinks only by $1 - \omega\lambda_{\min}(S) = 0.99982$ per sweep, a factor of 0.025 over the whole budget. Steepest descent is limited similarly: at most $(\kappa - 1)/(\kappa + 1)$ per step gives about 0.012 over 20,000 steps. Gauss–Seidel meets the target after 19,136 sweeps; its long time mainly reflects one sparse triangular solve per sweep in this implementation.

The Krylov methods need 226–242 iterations. Bound (12) guarantees an energy-norm reduction by 10^{-8} within $\frac{1}{2}\sqrt{\kappa}\ln(2 \times 10^8) \approx 912$ CG iterations; CG needs about a quarter of this, because the bound uses only the extreme eigenvalues [12]. Diagonal preconditioning helps little: $\kappa_2(S) = 8029$ is only 12% below $\kappa_2(A)$, since the diagonal of A varies only near the boundary, and PCG saves 16 iterations. Full GMRES needs the fewest iterations but is 35 times slower

than CG, the cost of orthogonalizing against all previous basis vectors [6]. At this size, sparse LU factorization and solve together take 2.3 ms, less than any iterative method.

Equal residual targets do not give equal accuracy: the errors of Gauss–Seidel and CG differ by a factor of 180, both within the bound $\kappa_2(A)\,\eta \approx 9.1 \times 10^{-5}$ from Eq. (14). Iterations are algebraic corrections towards one static equilibrium, not physical time steps.

4.3 Deformation and validation

Figure 2c shows the 20×20 solution. The right edge moves between 0.4916 mm (mid-height) and 0.4967 mm (bottom corner), close to $pL/E = 0.5$ mm. Near the loaded end, the top and bottom edges move inwards by 0.077 and 0.079 mm, together matching the free lateral contraction $\nu pL/E = 0.15$ mm. The clamp prevents this contraction at $x = 0$, producing the tapered shape.

For validation, all 42 prescribed components are exactly zero in the reconstructed d. The reactions of Eq. (7) sum to -1.000×10^6 N in x, leaving a force imbalance of $(-4.0, -2.6) \times 10^{-9}$ N and a moment imbalance about the origin of 4.5×10^{-9} N m, against an applied moment of 5×10^5 N m. The five methods that met the target agree with LU to at most 1.7×10^{-7} relative, or 8.3×10^{-8} mm in any component. On the same mesh, the roller-supported patch test of Section 3 (860 unknowns) reproduces the exact field $u_x = px/E$, $u_y = -\nu py/E$ with relative error 2.6×10^{-14}, verifying assembly, loading and reduction. The clamped plate has no closed-form solution, so Section **??** estimates its discretization error by comparing meshes from 5×5 to 80×80 cells, together with the growth of $\kappa_2(A)$ and the iteration counts.

5 Additional objects and experimental design

The square of Section 4 is stretched almost uniformly, has a closed-form check and uses a structured mesh. Typical parts instead have openings, corners and slender members, and can be supported in different ways. These features raise questions the square cannot answer: how shape concentrates deformation and stress, whether geometry and supports change the conditioning of A and hence solver cost, and whether conclusions from structured meshes carry over to general triangulations. We therefore introduce four objects and specify comparisons that each change one factor while fixing the rest. Section **??** reports their outcomes.

5.1 Objects and meshes

Table 2 and Fig. 4 define the objects; all four are selected experiments, not illustrations. The square `S` repeats Section 4 with a different mesh generator, so that differences from the structured results reflect the mesh alone. The plate `H` adds a central circular hole, the classical source of stress concentration: in a wide plate under uniaxial tension, the stress at the edge of a small hole reaches three times the applied value [**?**]. The hole also makes the region multiply connected. The L-bracket `L` has a re-entrant corner, where the boundary turns through 270° and the continuum stress is unbounded [**?**]; displacements should converge under refinement there, but local stresses cannot. Its two arms have free end faces, so the load can be moved without changing A. The beam `B` is slender and bends under a tip load. Beam theory then gives an independent estimate of its deflection, while linear triangles are known to be too stiff in bending on coarse meshes [**?**].

Each boundary is an ordered vertex list whose segments carry a label: clamped, loaded or free. We split every segment into equal pieces of length at most h and pass the result to Triangle [11] through its Python interface [**?**] with switches `p` (keep the segments), `q30` (no angle below 30°) and `a` (area at most $\sqrt{3}h^2/4$, an equilateral triangle of side h). This constrained Delaunay mesh keeps every boundary segment as a union of triangle edges, and subsegments inherit the labels. The hole is a regular polygon with $4\lceil 2\pi R/(4h) \rceil$ vertices on the circle plus a seed point at its centre, from which Triangle removes all triangles. The points where we compare displacements, marked in Fig. 4, are input vertices and hence nodes on every level. Four levels

Table 2: Objects: ordered counter-clockwise outer vertices, openings, supports and loads. Every object is clamped ($u_x = u_y = 0$) on its edge $x = 0$. N is the number of unknowns on mesh levels 1–4.

Object	Outer boundary (m)	Opening	Baseline load	N, levels 1–4
S square	$(0,0), (1,0), (1,1), (0,1)$	–	$(p, 0)$ on $x = 1$	NS
H hole plate	as S	disc, centre $(0.5, 0.5)$, $R = 0.15$	$(p, 0)$ on $x = 1$	NH
L bracket	$(0,0), (1,0), (1,0.5), (0.5,0.5), (0.5,1), (0,1)$	–	$(p, 0)$ on $x = 1$	NL
B beam	$(0,0), (4,0), (4,0.5), (0,0.5)$	–	$(0, -\tau)$ on $x = 4$	NB

use $h = 0.1$, 0.05, 1/30 and 0.025 m for S, H and L, and $h = 0.125$, 0.0625, 1/24 and 0.03125 m for B, i.e. 4 to 16 elements through its depth.

We check every mesh before use: all triangles have positive area in counter-clockwise order and no angle below 30°; their areas sum to the polygon area up to rounding; the edges used by only one triangle are exactly the boundary subsegments; no triangle centroid lies in the hole; and the mesh is connected with Euler characteristic $n_{\text{nodes}} - n_{\text{edges}} + n_{\text{triangles}} = 1$ minus the number of holes. All meshes below pass. The polygonal hole is slightly larger than the disc; this geometric difference vanishes as $h \to 0$ and is part of the discretization error, not a meshing fault.

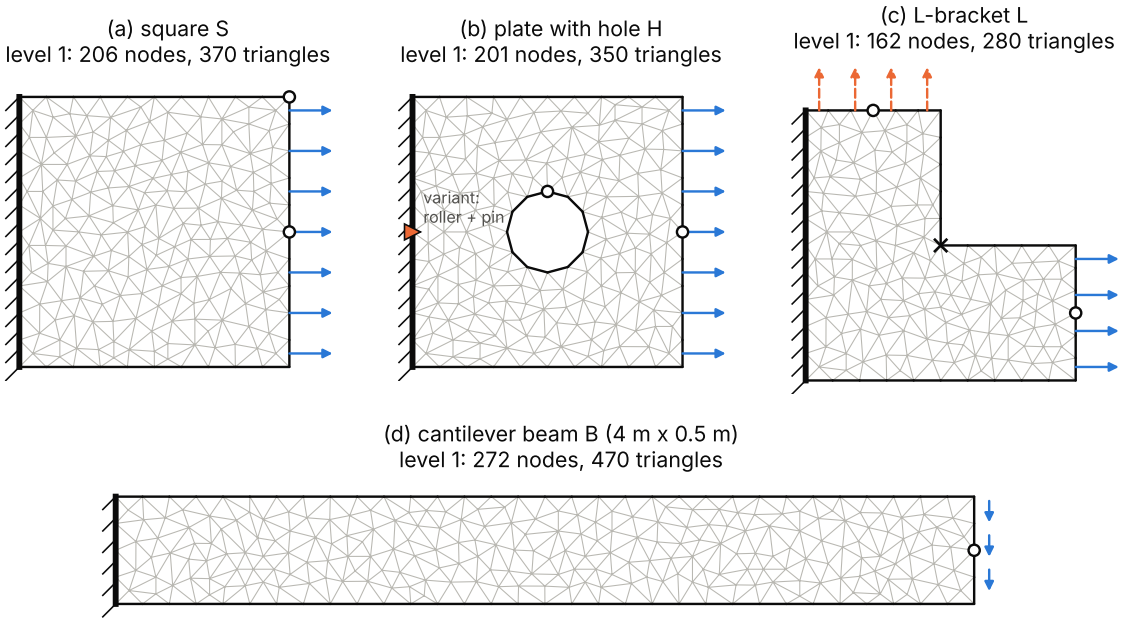


Figure 4: The four objects on their coarsest meshes (level 1). Hatched lines: clamped edges. Solid blue arrows: baseline traction. Dashed orange arrows: alternative load location on the bracket. Orange triangle: pin of the roller variant of the hole plate. Circles: matched points where displacements are compared across levels; cross: re-entrant corner.

5.2 Material, loads and supports

The baseline matches Section 4: plane stress, $E = 200$ GPa, $\nu = 0.3$, $t = 0.01$ m, no body forces, and the left edge clamped. The square, plate and bracket carry the uniform normal traction $p = 100$ MPa on their right end face, with resultant $pt\ell$ for face length ℓ: 10^6 N for S and H, 5×10^5 N for L. The beam carries the downward shear traction $\tau = 10$ MPa on its tip, giving $P = \tau t W_B = 5 \times 10^4$ N for depth $W_B = 0.5$ m; the smaller load keeps its bending strains of

order 10^{-3}. For length $L_B = 4$ m, beam theory predicts the tip deflection [**?**]

$$\delta \approx \frac{PL_B^3}{3EI} + \frac{PL_B}{\kappa_s G t W_B}, \qquad I = \frac{tW_B^3}{12}, \quad G = \frac{E}{2(1+\nu)}, \quad \kappa_s = \frac{5}{6}, \tag{16}$$

where the first (Euler–Bernoulli) term is 51.20 mm and the shear correction adds 0.62 mm. The clamp in our model also stops the root section from warping, which beam theory ignores.

Loads enter as consistent nodal forces, $\ell_e q/2$ per endpoint of each loaded edge of length ℓ_e (Section 2). Because loads and clamps are tied to labelled segments, every mesh level carries the same resultant on the same face and is held on the same edge: the physical problem is fixed under refinement. A clamped edge prevents both translations and the rotation, so each A should be positive definite. We confirm this for every reduced system: the smallest eigenvalue from shift-invert `eigsh` is positive, and dense Cholesky succeeds up to 3000 unknowns. As a negative control, restraining only u_x on the left edge leaves a free vertical translation; this check must then detect a singular A.

5.3 Case register

Table 3 lists the comparisons. Each changes one factor and states what is held fixed.

Table 3: Case register. Level 2 is $h = 0.05$ m (B: 0.0625 m). u_P: displacement at a matched point.

Study	Question	Changed factor	Fixed	Measured response
Geometry	How do a hole and a corner change deformation and A?	S, H, L	material, p, clamp, h	u_P, max $\lvert\mathbf{u}\rvert$, κ_2, iterations
Refinement	Do responses settle, and how fast do κ_2 and cost grow?	h, levels 1–4	object, load, supports	u_P, peak stress, κ_2, ρ_J, iterations vs N
Mesh type	Do structured-mesh results carry over?	square grid (n = 10–40) vs S	square problem	u_P, κ_2, CG/PCG iterations
Load magnitude	Is the response linear?	$p = 25, 100, 250$ MPa (H)	A	u_P/p, iterations
Load location	What does moving the load change?	face $x = 1$ vs $y = 1$ (L)	A, resultant	loaded-face u_P, iterations
Supports	How does the clamp shape the response and A?	clamp vs roller $u_x = 0$ + pin $u_y = 0$ at $(0, 0.5)$ (H)	load, mesh	u_P, κ_2, iterations
Material	How do E and ν enter?	$\nu = 0, 0.3, 0.45$; $E = 70,200$ GPa (H)	geometry, load	u_P, κ_2, iterations
Bending	Do linear triangles capture bending?	h, levels 1–4 (B)	beam, P	tip u_y vs Eq. (16), κ_2

The geometry study holds support, traction and mesh rule fixed, so differences in displacement and κ_2 come from shape alone; the bracket's resultant is half that of the square, which we account for when comparing displacements. The refinement study tests whether matched-point displacements settle, which estimates discretization error, and whether κ_2 grows like h^{-2} and CG iterations like $\sqrt{\kappa_2}$, as Section 3 predicts for quasi-uniform meshes. Peak element stresses at the hole edge and at the re-entrant corner show where local quantities converge and where they cannot. The mesh-type study compares S with Section 4's grids at $n = 10$, 20, 30 and 40 cells per side. On unstructured meshes the diagonal of A varies with element shape, which Section 3 identifies as the condition for diagonal preconditioning to help.

The remaining studies are designed so that the expected effect is known in part. Scaling the load scales b only, so x must scale with it and, since η is scale invariant, iteration counts should not change. The two bracket loads have equal resultants and share A, so any difference in iterations reflects the right-hand side alone. The roller and pin remove rigid motion with fewer constraints and let the left edge contract freely; this changes A itself. Changing E multiplies

D, and hence A, by a constant: x scales as $1/E$ while κ_2 and every iteration count stay fixed in exact arithmetic, which tests the implementation. Changing ν alters the ratios of entries in D and can change the spectrum. Finally, the beam tip deflection is compared with Eq. (16), and an axial variant with traction $(p, 0)$ on the same level-2 mesh, sharing its A, is compared with the bar estimate $pL_B/E = 2$ mm.

Every reduced system is solved by sparse LU, CG, diagonal PCG, MINRES and Gauss–Seidel under the protocol of Section 3; the level-2 baseline of each object also receives all eight iterative methods. We record iterations, η, error against LU, times, κ_2, ρ_J, stored nonzeros and N. Because a Gauss–Seidel run takes up to about 20 s here, it is timed once, without a separate history run, except on levels 1–2 of the baseline cases. Each solution is validated by the force and moment balance of applied loads and reactions (Eq. (7)) and by the prescribed values $d_C = 0$. The script `draft/experiments/objects.py` generates all meshes, systems and outputs.

References

[1] Jeremy Bleyer, *Isotropic and orthotropic plane stress elasticity*, Numerical Tours of Computational Mechanics with FEniCSx, `https://bleyerj.github.io/comet-fenicsx/tours/linear_problems/isotropic_orthotropic_elasticity/isotropic_orthotropic_elasticity.html`. Accessed 10 November 2026.

[2] Howard C. Elman, David J. Silvester, and Andrew J. Wathen, *Finite elements and fast iterative solvers: With applications in incompressible fluid dynamics*, second ed., Oxford University Press, Oxford, 2014.

[3] Anders Logg and Hans Petter Langtangen, *The equations of linear elasticity*, FEniCSx tutorial, maintained by Jørgen Schartum Dokken, Accessed 10 November 2026.

[4] C. C. Paige and M. A. Saunders, *Solution of sparse indefinite systems of linear equations*, SIAM Journal on Numerical Analysis **12** (1975), no. 4, 617–629.

[5] Youcef Saad and Martin H. Schultz, *GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems*, SIAM Journal on Scientific and Statistical Computing **7** (1986), no. 3, 856–869.

[6] Yousef Saad, *Iterative methods for sparse linear systems*, second ed., SIAM, Philadelphia, 2003.

[7] SciPy community, *`scipy.sparse.linalg.eigsh`*, SciPy Reference Guide, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.eigsh.html`. Accessed 10 November 2026.

[8] ———, *`scipy.sparse.linalg.gmres`*, SciPy Reference Guide, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.gmres.html`. Accessed 10 November 2026.

[9] ———, *`scipy.sparse.linalg.minres`*, SciPy Reference Guide, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.minres.html`. Accessed 10 November 2026.

[10] ———, *`scipy.sparse.linalg.splu`*, SciPy Reference Guide, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.splu.html`. Accessed 10 November 2026.

[11] Jonathan Richard Shewchuk, *Triangle: Generating Delaunay triangulations*, Carnegie Mellon University, Triangle documentation, Accessed 10 November 2026.

[12] ———, *An introduction to the conjugate gradient method without the agonizing pain*, Technical Report CMU-CS-94-125, School of Computer Science, Carnegie Mellon University, 1994, Edition $1\frac{1}{4}$.

[13] Lloyd N. Trefethen and David Bau, III, *Numerical linear algebra*, SIAM, Philadelphia, 1997.

[14] University of Alberta, *Two dimensional solid elements: Triangular elements*, Introduction to Solid Mechanics, Engineering at Alberta Courses, Open Educational Resources, `https://engcourses-uofa.ca/books/introduction-to-solid-mechanics/finite-element-analysis/two-dimensional-solid-elements/triangular-elements/`. Accessed 10 November 2026.